RAGOps

2-Person Resume-Level Project Plan

Goal: Build a focused RAG experimentation and evaluation platform that is strong enough for a resume and technical interviews, without turning the project into a startup-scale platform.

1. Project Objective

RAGOps will allow users to upload documents, build a searchable knowledge base, ask questions, and compare different RAG retrieval configurations. The main differentiator is measurable comparison of retrieval quality rather than simply building another PDF chatbot.

Core comparison: Baseline RAG → Dense Retrieval → Hybrid Retrieval → Hybrid + Reranker

2. Final MVP

Feature	Required
PDF document upload	Yes
Text extraction + cleaning	Yes
Chunking	Yes
Embeddings	Yes
PostgreSQL + pgvector	Yes
Dense retrieval	Yes
BM25 / keyword retrieval	Yes
Hybrid retrieval	Yes
Reranking	Yes
LLM answer generation	Yes
Citations / source chunks	Yes
Retrieval + generation evaluation	Yes
Experiment comparison dashboard	Yes
Docker + deployment	Final stage

3. Team Structure

Both members work across ML/RAG and development. Each week has a lead owner, but the second member reviews and contributes to the same deliverable. This prevents the project from becoming two disconnected halves.

Member A — Primary Strength	Member B — Primary Strength
RAG and retrieval Embeddings BM25 Hybrid retrieval Reranking Evaluation Experiment analysis	FastAPI PostgreSQL + pgvector React API integration Dashboard Docker/deployment
Cross-training: ~30% development contribution	**Cross-training:** ~30% ML/RAG contribution

4. Six-Week Development Plan

Week 1 — Foundation

Member A: RAG, embeddings, vector similarity, dense retrieval, BM25, reranking, evaluation basics.
Member B: FastAPI, PostgreSQL, pgvector, React, API design.
Both: Architecture, schema, GitHub workflow, dataset and stack.
Deliverable: Architecture + repository skeleton + DB connection.

Week 2 — Document Ingestion

Member A: PDF extraction → cleaning → chunking → embeddings; integrate processing with backend.
Member B: Upload API, file handling, metadata, pgvector storage and upload UI.
Deliverable: PDF → chunks → embeddings → PostgreSQL/pgvector.

Week 3 — Basic RAG MVP

Member A: Dense retrieval, top-K, context construction, prompt, LLM generation and citations.
Member B: Query API, frontend query page, response display and integration.
Deliverable: Upload → retrieve → generate → answer + citations.

Week 4 — Retrieval Experiments

Member A: BM25, hybrid retrieval and initial reranking; experiment methodology.
Member B: Experiment API, configuration storage and comparison UI.
Deliverable: Dense vs BM25 vs Hybrid comparison.

Week 5 — Evaluation

Member A: Recall@K, Precision@K, MRR, NDCG; retrieval evaluation API.
Member B: Faithfulness, answer relevance, context relevance, citation accuracy; generation evaluation UI/API.
Both: Review each other's evaluation code and run the shared benchmark.
Deliverable: Evaluation dashboard.

Week 6 — Finalization

Member A: Final benchmarks, ML analysis, retrieval optimization and graphs.
Member B: Dashboard polish, Docker, deployment, README and demo flow.
Both: Report, presentation, architecture explanation, viva preparation and resume bullets.
Deliverable: Deployed, documented and benchmarked RAGOps.

5. Target Architecture

React → FastAPI → PostgreSQL + pgvector

Ingestion: PDF → extraction → cleaning → chunking → embeddings
Retrieval: Dense / BM25 / Hybrid → optional Reranker
Generation: LLM + source citations
Evaluation: Retrieval metrics + generation metrics
Dashboard: Compare experiments and visualize results

6. Recommended Tech Stack

Layer	Technology	Purpose
Frontend	React	Upload, query and dashboard
Backend	FastAPI	REST APIs and orchestration
Database	PostgreSQL + pgvector	Metadata + vector storage
RAG	LangChain or LlamaIndex	Pipeline integration
Embeddings	One strong embedding model	Semantic representation
Retrieval	Vector search + BM25	Dense, keyword and hybrid
Reranking	Cross-encoder	Improve context ordering
LLM	One API/cloud LLM	Answer generation
Deployment	Docker + suitable cloud	Reproducible deployment

7. Final Benchmark

Run the same evaluation dataset through four configurations:

Configuration	Purpose
Baseline RAG	Reference point
Dense RAG	Semantic retrieval performance
Hybrid RAG	Dense + keyword retrieval
Hybrid + Reranker	Best-quality retrieval candidate

Metrics: Recall@K, Precision@K, MRR/NDCG, faithfulness, answer/context relevance, citation accuracy and latency.

8. What NOT to Build

• Microservices architecture

• Kubernetes

• Custom LLM training/fine-tuning

• Complex authentication and role systems

• Large-scale distributed vector infrastructure

• Over-engineered experiment orchestration

• Huge failure taxonomy

Target: a technically credible, measurable and deployable student project — not a startup platform.

9. Resume Positioning

Suggested positioning: Built a full-stack RAG experimentation platform that ingests documents, supports dense/BM25/hybrid retrieval with reranking, evaluates retrieval and generation quality, and visualizes benchmark results through a web dashboard.

Interview topics: Why hybrid retrieval? When does reranking help? Recall@K vs MRR? How was faithfulness evaluated? What latency/quality trade-off appeared? Which configuration performed best and why?

10. Working Rules

• Commit small, understandable changes.

• Use GitHub issues/tasks so ownership is clear.

• A feature is complete only after end-to-end integration and testing.

• Both members must understand the complete RAG pipeline.

• Use one shared evaluation dataset for comparable experiments.

• Prioritize working functionality over UI polish.

• If a feature does not improve learning, resume value or the final demo, cut it.